

Part 2 – Infrastructure Design

Requirements

The infrastructure should provide:

- Auto-scaling
- Secure access
- Reasonable cost

Proposed Infrastructure Architecture

I Recommend Compute Choice – GKE (Google Kubernetes Engine)

Why GKE?

GKE is recommended because:

- Supports auto-scaling
- Easy rolling deployments
- Better container orchestration
- High availability
- Easier future scaling
- Supports self-healing workloads

Why Not Compute Engine?

Compute Engine VMs require:

- Manual scaling setup
- More operational overhead
- Manual orchestration
- Complex deployments

Why Not Cloud Run?

Cloud Run is serverless and cost-effective, but:

- Less control over infrastructure

- Not ideal for long-running enterprise backend workloads
- Limited customization compared to Kubernetes

MongoDB Hosting Approach

Recommended Option – MongoDB Atlas

Why MongoDB Atlas?

- Managed database service
- Automated backups
- Built-in scaling
- High availability
- Reduced operational effort
- Easy integration with GCP

Secure Connectivity

Connection between GKE and MongoDB Atlas secured using:

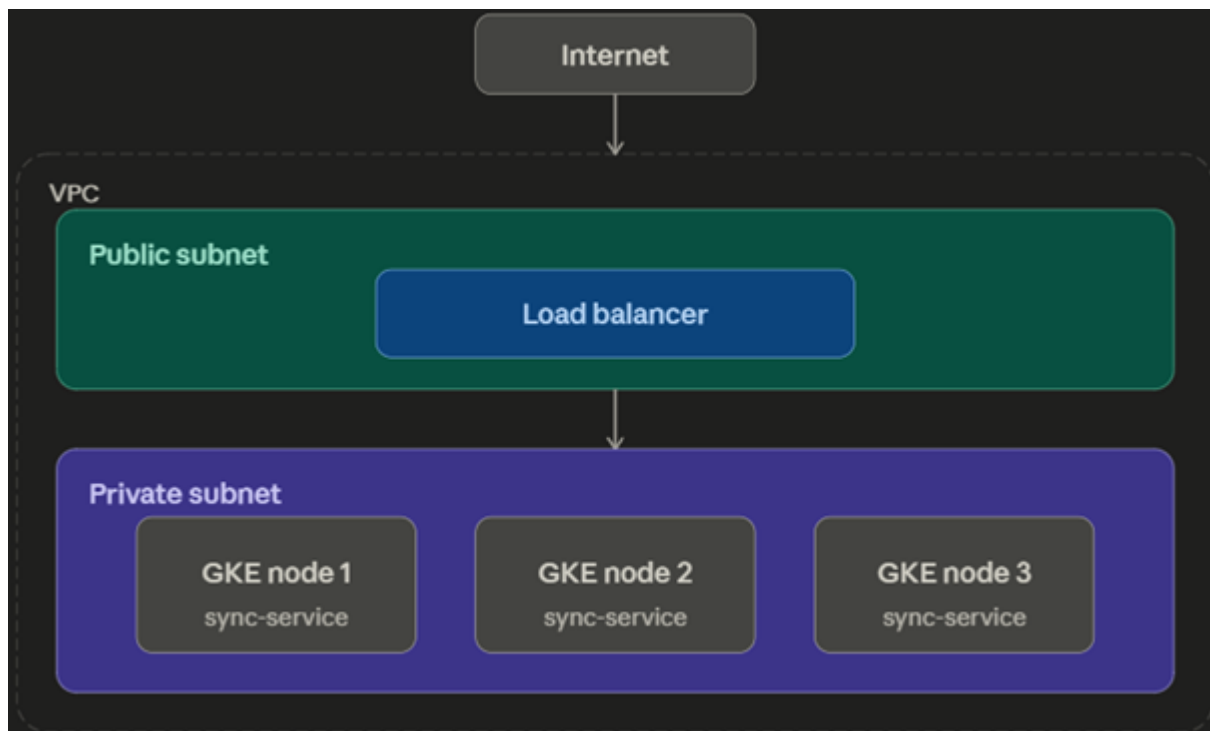
- Private networking
- IP whitelisting
- TLS encryption

Networking Design

VPC Setup

A dedicated VPC should be created.

I Will follow the below structure:



Ingress

Will Use GCP Load Balancer plus Kubernetes Ingress.

Responsibilities:

- SSL termination
- Traffic routing
- Health checks
- External access management

Firewall Rules

Allow:

- HTTPS (443)
- Internal communication between services

Restrict:

- SSH access
- Direct database access from internet

Secrets & IAM

Secrets Management

Will Use GCP Secret Manager.

Secrets stored:

- MongoDB credentials
- API keys
- Tokens

Kubernetes workloads access secrets using:

- Workload Identity
- IAM Service Accounts

IAM Best Practices

I will Apply Principle of Least Privilege.

Examples:

Service	Access
Jenkins	Deploy permissions only
Application Pods	Secret access only
Developers	Read-only logs

Logging & Monitoring

I use the following Monitoring Stack

For I use Logging

- GCP Cloud Logging

For Monitoring

- GCP Cloud Monitoring
- Prometheus
- Grafana

Alerts

Will Configure alerts for:

- High CPU usage
- Pod failures
- Application downtime
- High response time
- Memory spikes

Conclusion

This solution provides:

- Secure CI/CD pipeline using Jenkins
- Controlled deployments across QA, staging, and production
- Secure secret management using GCP Secret Manager
- Auto-scaling infrastructure using GKE
- Minimal downtime deployments
- Centralized logging and monitoring
- Cost-optimized and production-ready architecture

The proposed setup is scalable, secure, maintainable, and aligned with real-world DevOps best practices.